

HackDNA – Secrets in Source 2

Challenge Scenario

SecureVault Technologies claims their website is protected with advanced client-side security controls. The application attempts to prevent inspection by:

- Disabling right-click
- Blocking developer tools
- Detecting browser inspection activity
- Restricting keyboard shortcuts

Despite these protections, client-side defenses alone cannot secure sensitive information exposed to the browser.

This challenge demonstrates why frontend security controls are not reliable protections against determined attackers.

Objective

Bypass the client-side restrictions and uncover hidden secrets exposed within the application's source code.

Understanding Client-Side Security Limitations

Anything delivered to the browser is ultimately accessible to the user.

Client-side protections such as:

- Disabled right-click
- Blocked keyboard shortcuts
- Anti-debugging scripts
- DevTools detection

only create superficial obstacles.

Attackers can still inspect:

- HTML source
- JavaScript files
- Network requests
- Browser storage

- Client-side variables
-

Initial Reconnaissance

Open the target application and observe its behavior.

Possible restrictions may include:

```
window.oncontextmenu = function() {  
    return false;  
}
```

or:

```
document.onkeydown = function(e) {  
    if (e.keyCode == 123) {  
        return false;  
    }  
}
```

These scripts attempt to block:

- Right-click
 - F12
 - CTRL + SHIFT + I
 - CTRL + U
-

Bypassing Client-Side Restrictions

Client-side restrictions can often be bypassed easily.

Method 1 – Browser Menu

Access developer tools using the browser menu:

```
Browser Menu → More Tools → Developer Tools
```

Method 2 – Disable JavaScript

Disable JavaScript temporarily.

Example in Chromium-based browsers:

```
Settings → Privacy and Security → Site Settings → JavaScript
```

Once JavaScript is disabled, anti-inspection protections stop functioning.

Method 3 – View Source Directly

Even if shortcuts are blocked, manually access:

```
view-source:http://TARGET
```

inside the browser address bar.

Method 4 – Use External Tools

Retrieve source code directly using:

```
curl http://TARGET
```

or:

```
wget http://TARGET
```

Client-side restrictions do not affect external HTTP requests.

Source Code Analysis

After bypassing restrictions, inspect the source carefully.

Look for:

- Hidden comments
- Hardcoded credentials
- API keys

- Tokens
- Hidden endpoints
- Debug information

Example:

```
<!-- TODO: remove admin password before production -->
```

or:

```
const adminKey = "SVT-admin-2026";
```

JavaScript Enumeration

Inspect linked JavaScript files.

Example:

```
<script src="app.js"></script>
```

Open browser DevTools:

```
F12 → Sources
```

Search for sensitive terms:

```
password  
secret  
admin  
apikey  
token  
flag
```

Hidden Flags and Secrets

Flags may be intentionally hidden inside:

- JavaScript variables
- Console messages
- HTML comments
- Local storage
- Hidden DOM elements

Example:

```
console.log("flag{client_side_security_fails}")
```

or:

```
<!-- flag{inspect_everything} -->
```

Browser Storage Inspection

Inspect local storage and session storage.

Open:

```
F12 → Application
```

Check:

- Local Storage
- Session Storage
- Cookies

Example:

```
flag=HDNA{source_disclosure}
```

Network Traffic Analysis

Inspect HTTP requests and responses.

F12 → Network

Look for:

- Hidden API responses
- Debug endpoints
- Sensitive JSON data
- Developer headers

Example response:

```
{
  "debug": "enabled",
  "admin": "true"
}
```

Root Cause

The vulnerability exists because sensitive information was exposed client-side while relying on weak browser restrictions as security controls.

Developers incorrectly assumed that:

- Blocking right-click prevents inspection
- Disabling shortcuts prevents analysis
- DevTools detection stops attackers

In reality, frontend protections cannot secure exposed data.

Security Impact

Source-code exposure vulnerabilities may lead to:

- Credential disclosure
- Administrative access
- API abuse

- Internal infrastructure exposure
- Authentication bypass
- Privilege escalation

Attackers routinely analyze frontend assets during reconnaissance.

Detection Techniques

Manual Analysis

- View source code
- Inspect JavaScript files
- Analyze browser storage
- Review network requests
- Check hidden elements

Automated Discovery

Using grep:

```
grep -Ri "password\|secret\|token\|apikey" .
```

Using curl:

```
curl http://TARGET
```

Using browser DevTools:

```
F12 → Sources
```

Defensive Measures

Never Trust Client-Side Controls

Client-side restrictions should never be considered security mechanisms.

Keep Secrets Server-Side

Sensitive information must never be embedded inside frontend code.

Remove Debug Information

Eliminate:

- Comments
- Debug logs
- Test credentials
- Development endpoints

before deployment.

Use Proper Authorization

Security decisions must always be enforced server-side.

Conduct Secure Code Reviews

Implement:

- Secret scanning
- Static analysis
- CI/CD security checks
- Penetration testing

Real-World Relevance

Many real-world applications attempt to rely on:

- Obfuscation
- Disabled right-click
- Anti-debugging JavaScript
- Minified code

These measures may inconvenience casual users but do not stop attackers.

Security through obscurity is not effective protection.

Key Takeaway

If the browser receives the data, attackers can access it.

Client-side protections cannot secure sensitive information exposed to users.

Vulnerability Classification

- CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
 - CWE-602: Client-Side Enforcement of Server-Side Security
 - CWE-215: Information Exposure Through Debug Information
 - OWASP A05:2021 – Security Misconfiguration
-

Conclusion

This challenge demonstrates the fundamental weakness of relying on client-side security controls. Browser restrictions may slow inexperienced users, but they do not protect sensitive information from attackers.

Secure applications must keep sensitive logic and secrets server-side while treating all client-side code as publicly accessible.